COMP6237 Data Mining
Lecture 5: Search and Rank

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

Lecturer (Assistant Professor) @ VLC of ECS

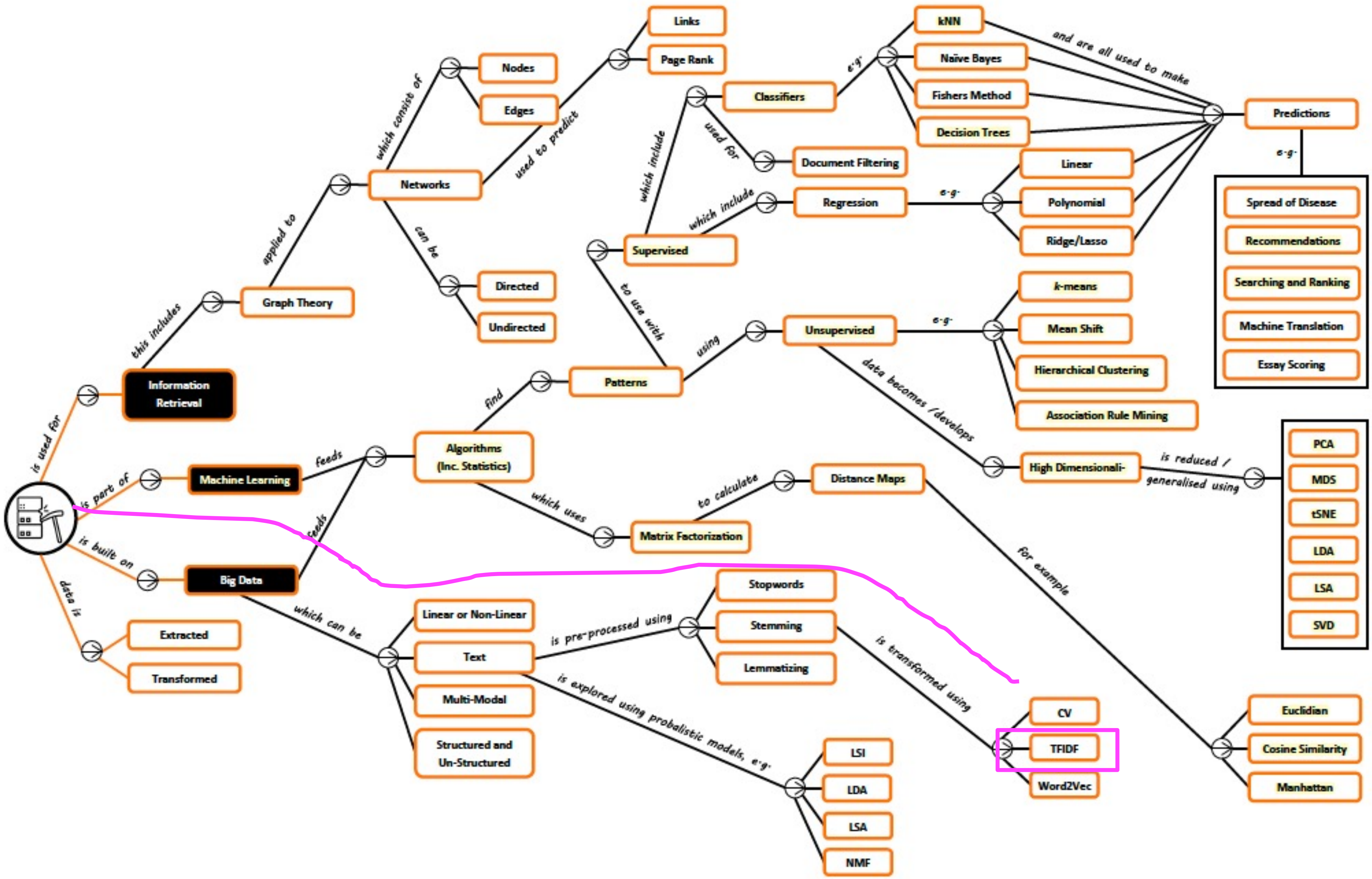
University of Southampton

Lecture slides available here:

http://comp6237.ecs.soton.ac.uk/zh.html

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

Search and Rank – **Roadmap**



Search and Rank – **Context**

University of Southampton

Google

search and ranking problem

AI Mode | **All** | Images | Videos | Short videos | Forums | News | More | Tools

AI Overview

The search and ranking problem in information retrieval is **the process of identifying relevant documents (search) from a large dataset for a user query and ordering them (ranking) by relevance to maximize user satisfaction**. It involves mapping a query and document to a numerical score, often using machine learning to balance relevance, user intent, and content quality.

Key aspects of this problem include:

- **Two-Phase Architecture:** Modern systems typically use a retrieval phase to find relevant candidates, followed by a ranking phase (often using Learning to Rank (LTR) algorithms) to order them.

Show more

How Does Google Determine Ranking Results - Google Search
Usability of content. Our systems also consider the usability of content. When all other signals...
Google

what-is-ai-search-ranking - Algolia
26 Feb 2024 — The top ranked results get the highest engagement; conversely, when people...
Algolia

Wikipedia
https://en.wikipedia.org › wiki › Ranking_(information...
Ranking (information retrieval)
Ranking of query is one of the fundamental **problems** in information retrieval (IR), the scientific/engineering discipline behind **search** engines. Read more

Towards Data Science
https://towardsdatascience.com › Latest
How to Evaluate Search Relevance and Ranking
30 May 2024 — This article explores the **key metrics used for evaluating Search Relevance and Ranking**, empowering you to optimize your Search Engine and deliver a superior ... Read more

Search and Rank – **Problem**

Searching the web has become the default way to find information. However, there is so much information on the web, how can we find what we are actually looking for?

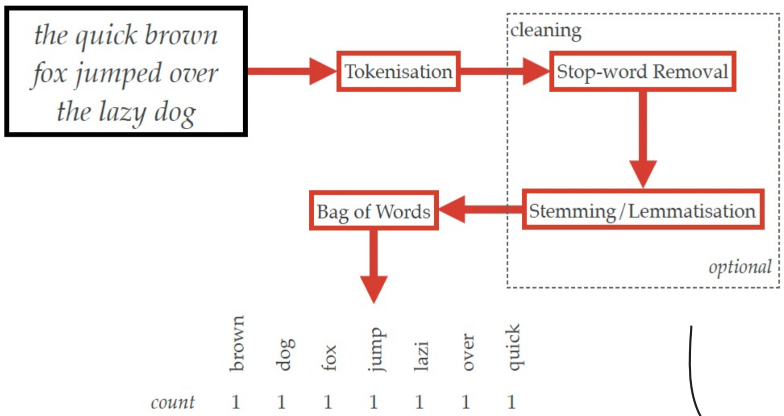
This is **information retrieval**

The goal of search and rank is to retrieve relevant information from a large collection of data based on a user's query and present the most relevant and useful results, ranked in order of importance.

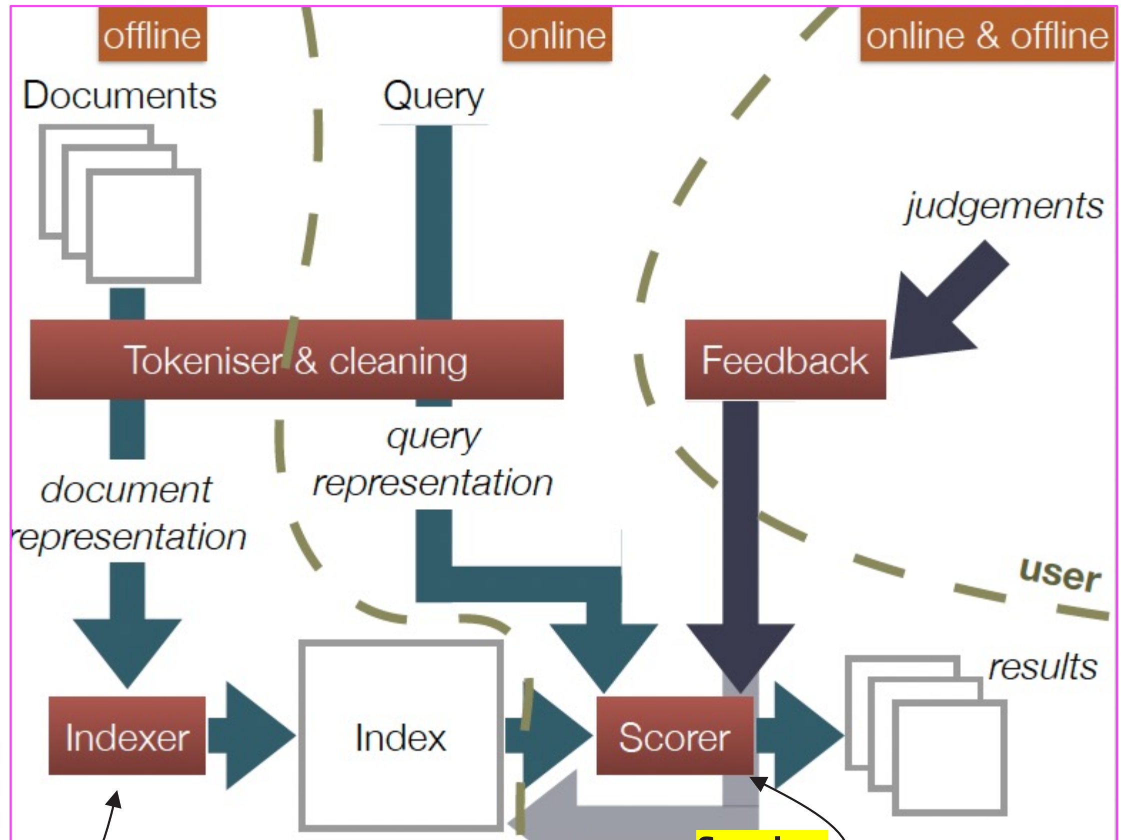
Searching the web is still the default way to find information.
But today, we don't just get *links* — we get *answers*.

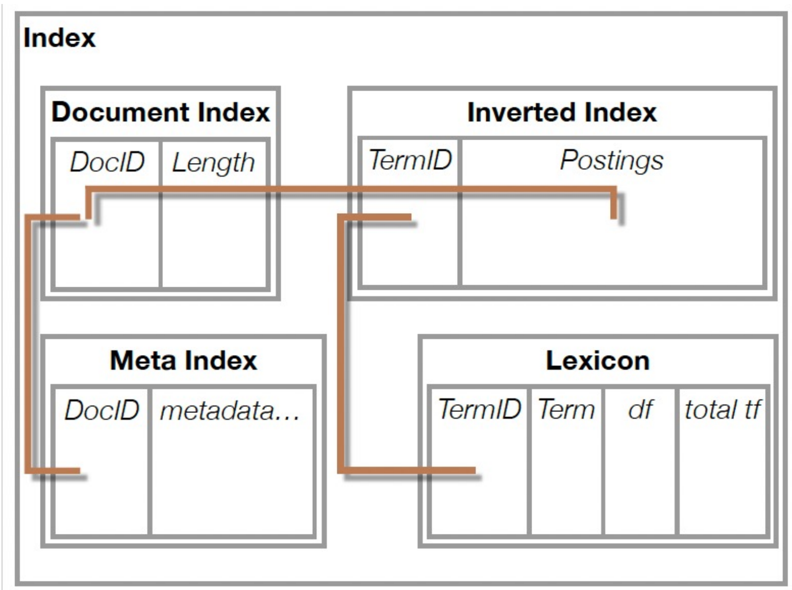
So the problem becomes:
How do systems decide what information is relevant — and in what order? This is still **information retrieval**.

Search and Rank – **Idea**



Encoding
(vector space modeling)





Indexing
(sorting)

Scoring
(matching)

Total #docs in collection

Number of times *w* appears in *d*

$$f(\mathbf{q},\mathbf{d}) = \sum_{i=1}^{N} q_i y_i = \sum_{w \in q \cap d} c(w,q)c(w,d)log\frac{M+1}{df(w)}$$

Number of times *w* appears in *q*

document frequency (number of docs containing *w*)

Search and Rank – Text Retrieval

Text retrieval:

- For a collection of text documents - text corpus
- User provides query - expresses information need
- Search engine returns relevant documents

This is **search technology** in industry

Search and Rank – Text Retrieval vs. database retrieval

	Database	Text
Information	Well-defined structure and semantics	Unstructured, ambiguous semantics
Query	Well defined semantics, complete specification (eg SQL)	Ambiguous, incomplete specification
Answers	Matched records	Relevant documents

You cannot prove mathematically what the best ways to retrieve a text item is. It is *empirically defined*, noone knows what the user wants until the user finds it.

Search and Rank – Text Retrieval Models

- Boolean Model: get every document that satisfies a Boolean Expression result selection
- Vector Space Model: how similar is document to query vector? ranked results
- Probabilistic Model: what is the probability that the document is generated by the query? ranked results

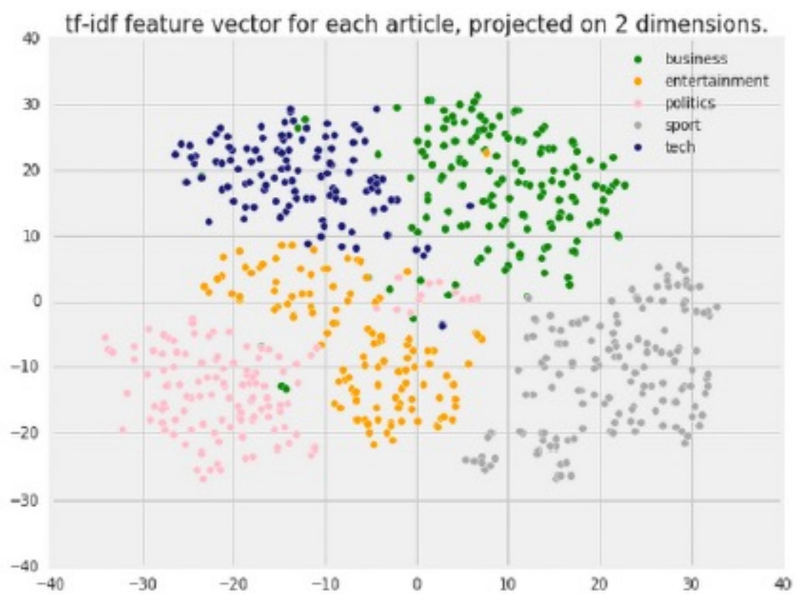
Selecting results is hard: if query is *over-constrained*, you may get nothing. If query is *under-constrained*, you may get way too many unsorted results.

Ranking is preferred, allows prioritisation.

Search and Rank – Vector Space Model

Vector Space Model is quite simple.

- Each document is a vector
- Each query is a vector
- Assume that if close together in space they are similar
- Rank each document by similarity



https://cloud.google.com/blog/products/gcp/problem-solving-with-ml-automatic-document-classification

Search and Rank – Vector Space Model

Each document is represented by a **term vector**
A **Term** is a basic concept, e.g. word or phrase
This gives an N-dimensional space for N **terms**

- Query Vector q $= (x_1, x_2, \ldots, x_N)$, $x_i \in \Re$ is query term weight
- Document Vector d $= (y_1, y_2, \ldots, y_N)$, $y_i \in \Re$ is document term weight

relevance$(\boldsymbol{q}, \boldsymbol{d}) \propto$ similarity$(\boldsymbol{q}, \boldsymbol{d}) =$ f$(\boldsymbol{q}, \boldsymbol{d})$

Search and Rank – Vector Space Model

As seen before, each **term** is assumed to be orthogonal
We still don't know:

- Which basic concepts to select for the **terms**
- How to assign weights $(x_1, x_2, \ldots, x_N)$ and $(y_1, y_2, \ldots, y_N)$
- How to define the similarity measure $(f(\boldsymbol{q}, \boldsymbol{d}))$

Quick Recap - Documents

e.g. "The quick brown fox jumped over the lazy dog."
becomes:

Tokenisation
'The': 1, 'quick': 1, 'brown': 1, 'fox': 1, 'jumped': 1, 'over': 1, 'the': 1, 'lazy': 1, 'dog.': 1

After further tokenising and sorting, you could get:
'brown': 1, 'dog': 1, 'fox': 1, 'jumped': 1, 'lazy': 1,'over': 1, 'quick': 1, 'the': 2

Further stemming and removal of stop words could give:
'brown': 1, 'dog': 1, 'fox': 1, 'jump': 1, 'lazi': 1,'over': 1, 'quick': 1

Quick Recap - Documents

The bag of words vector for a document will be very sparse

The vocabulary or lexicon will be the *set* of all (processed) words across all documents known to the system.

For a document, the vector contains the number of occurrences of each **term**, like a histogram

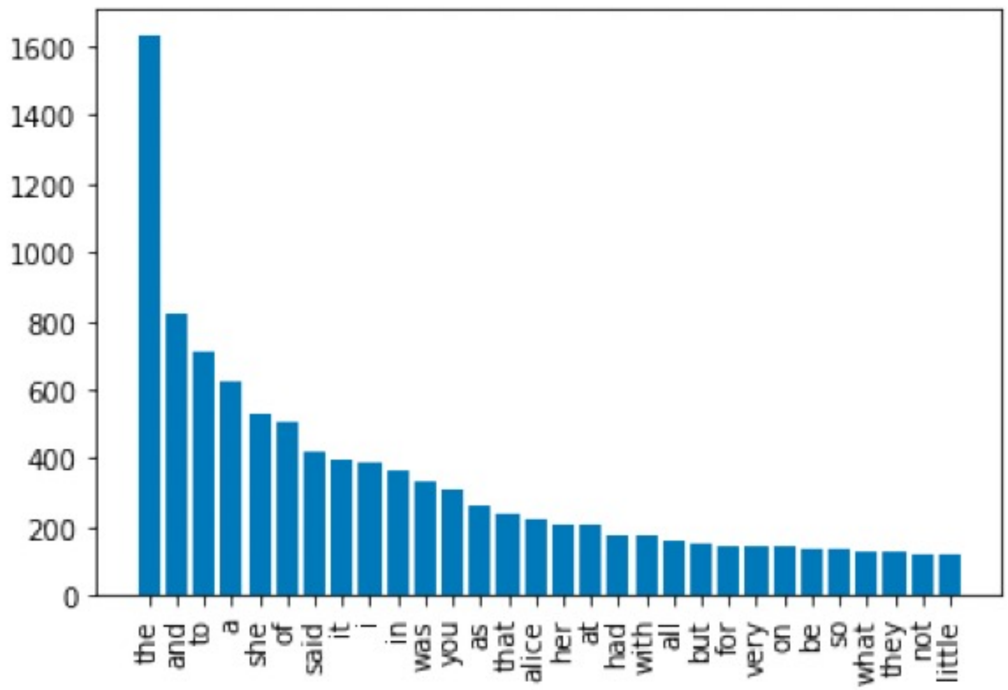
Vectors will have very high number of dimensions, but are very sparse
Alice ipynb demo

https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19

Search and Rank – Vector Space Model

Zipf's law

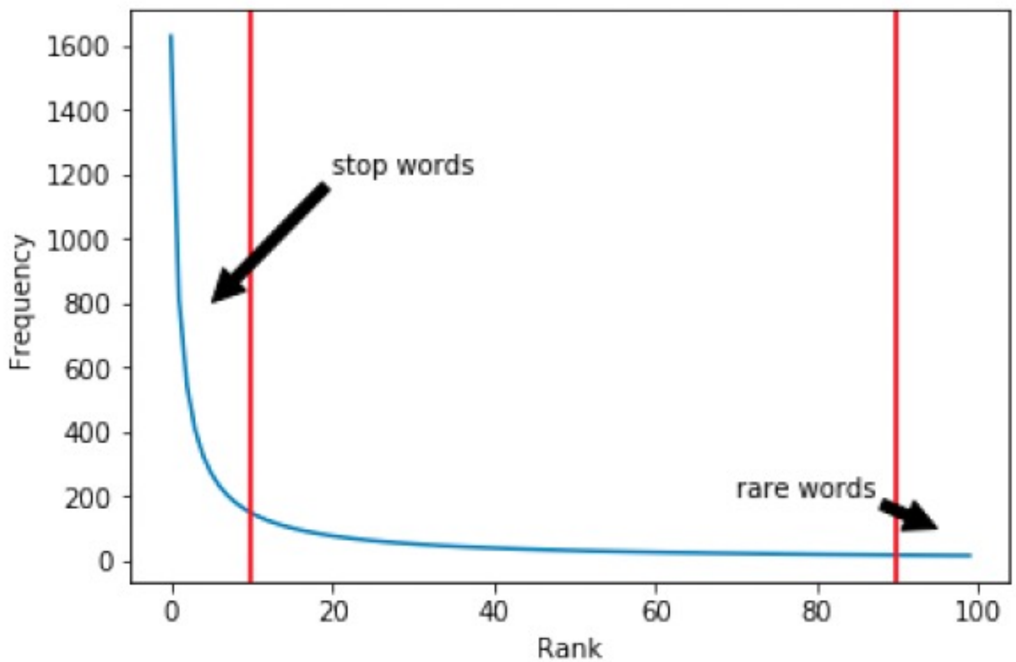
States that the frequency of a word is proportional to the inverse of its rank, i.e. the second most common word will be half the frequency of the first, the third will be a third the frequency of the first, e.t.c.



https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19

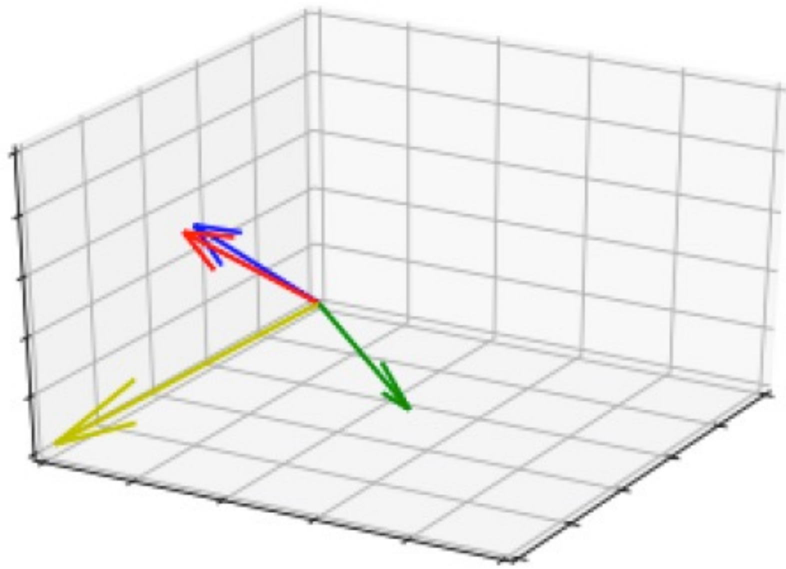
Search and Rank – Vector Space Model

We should remove the most common and the least common, as they hold little information, and may skew any vector to look similar.
We should also remove the very rare words, as they would make the document vector unnecessarily sparse without gaining much information.



Search and Rank – Vector Space Model

How to search the Vector Space Model?



If my query vector is red, which of the three document vectors is it closest to?

Search and Rank – Indexing

Inverted Index:
A mapping from content to location, e.g. in a set of documents.

Inverted index ipynb demo

alic	(0, 399), (1, 207), (2, 232)
said	(0, 462), (1, 11), (2, 0)
cooper	(0, 0), (1, 398), (2, 0)
spring	(0, 0), (1, 2), (2, 218)
not	(0, 145), (1, 21), (2, 5)
retriev	(0, 0), (1, 111), (2, 47)
littl	(0, 128), (1, 4), (2, 3)
one	(0, 105), (1, 21), (2, 5)

A posting is a pair formed by a document ID and the number of times the specific word appeared in that document.

Search and Rank – Indexing

To efficiently compute the cosine similarity, look up the relevant postings list and accumulate similarities only for the documents in those lists.

alic	(0, 399), (1, 207), (2, 232)
said	(0, 462), (1, 11), (2, 0)
cooper	(0, 0), (1, 398), (2, 0)
spring	(0, 0), (1, 2), (2, 218)
not	(0, 145), (1, 21), (2, 5)
retriev	(0, 0), (1, 111), (2, 47)
littl	(0, 128), (1, 4), (2, 3)
one	(0, 105), (1, 21), (2, 5)

For example: "Alice Cooper"

Accumulation table:

Doc ID	Frequency
0	399
1	207
2	232

Search and Rank – Indexing

To efficiently compute the cosine similarity, look up the relevant postings list and accumulate similarities only for the documents in those lists.

alic	(0, 399), (1, 207), (2, 232)
said	(0, 462), (1, 11), (2, 0)
cooper	(0, 0), (1, 398), (2, 0)
spring	(0, 0), (1, 2), (2, 218)
not	(0, 145), (1, 21), (2, 5)
retriev	(0, 0), (1, 111), (2, 47)
littl	(0, 128), (1, 4), (2, 3)
one	(0, 105), (1, 21), (2, 5)

For example: "Alice Cooper"

Accumulation table:

Doc ID	Frequency
0	399
1	207 + 398
2	232

cosine similarity ipynb demo

Search and Rank – Indexing

Using frequency of a word in a document is not always a good idea

How can we weight these vectors better?

- Binary weights
  record only if a word is present or absent in the vector
- Raw Frequency
  record frequency of occurrence of a term in the vector
- TF-IDF - Term Frequency - Inverse Document Frequency
  - Term Frequency - the raw frequency of a word in a document usually normalised by the number of words in the document
  - Inverse Document Frequency - 1/number of occurrences of word in all documents

  A high weight in TF-IDF is reached by a high frequency in a given document, but low frequency in the whole collection of documents, this would filter out the more common terms.

Search and Rank – Indexing

There are many variants of TF-IDF

For the TF (term frequency term:

- term frequency $\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$
- log normalisation $\log(1 + f_{t,d})$
- double normalisation K $K + (1-K)\frac{f_{t,d}}{\max_{t' \in d} f'_{t,d}}$

For the inverse document frequency term:

- Inverse document frequency $\mathrm{idf}(t,D) = \log \frac{N}{n_t}$
- Inverse document frequency smooth $\log \frac{N}{1+n_t}$
- Inverse document frequency max $\log \frac{\max_{t' \in d} n_{t'}}{i+n_t}$
- Probabilistic inverse document frequency $\log \frac{N-n_t}{n_t}$

where N is total number of documents in the corpus $N = |D|$

$n_t = \quad |\{d \in D : t \in d\}|$: number of documents where the term t appears (i.e., $\mathrm{tf}(t,d) \neq 0$).

Search and Rank – Indexing

Then TF-IDF is calculated as TF $\times$ IDF

TF-IDF ipynb demo

Some possible schemes for TF-IDF:

Document	*Query*	
$f_{t,d} \log \dfrac{N}{n_t}$	$(K + K \frac{f_{t,q}}{\max_t f_{t,q}}) \log \frac{N}{n_t}$	$K = 0.5$
$(1 + \log f_{t,d}) \log \dfrac{N}{n_t}$	$(1 + \log f_{t,q}) \log \frac{N}{n_t}$	

University of Southampton

Search and Rank – Matching

- Key bit of the cosine similarity is the dot-product between the query and document
  - Use this as a basis for building a scoring function that incorporates TF and IDF

Total #docs in collection

Number of times *w* appears in *d*

$$f(\mathbf{q}, \mathbf{d}) = \sum_{i=1}^{N} q_i y_i = \sum_{w \in q \cap d} c(w,q)c(w,d)log\frac{M+1}{df(w)}$$

Number of times *w* appears in *q*

document frequency (number of docs containing *w*)

$$\mathrm{idf}(t, D) = \log \frac{N}{n_t}$$

$$\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Search and Rank – Web Search

For web pages, we can use the number of links to a document as a way of scoring them

However this is prone to manipulation, as you can make lots of pages that all point to each other lots of times.

Page and Brin: PageRank Markus will cover this in May

In brief: Calculates the importance of a page from the importance of all the other pages that link to it and the number of links each of those other pages have.

Search and Rank – User Feedback

What user feedback do we get with a web search?

Use what they actually click on. Documents that are more clicked on could be more important.
Can also learn associations between queries and documents, and if this query is met again, use this to increase the rank of the document that was clicked on before for this query.

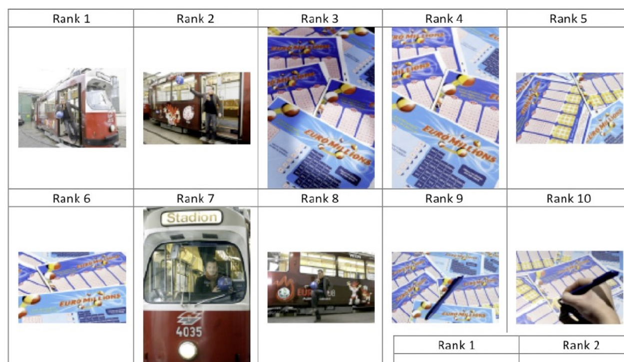
Search and Rank – Result Diversification

Is a ranked list always the right thing?

e.g. If I search for "University"
Do I want..

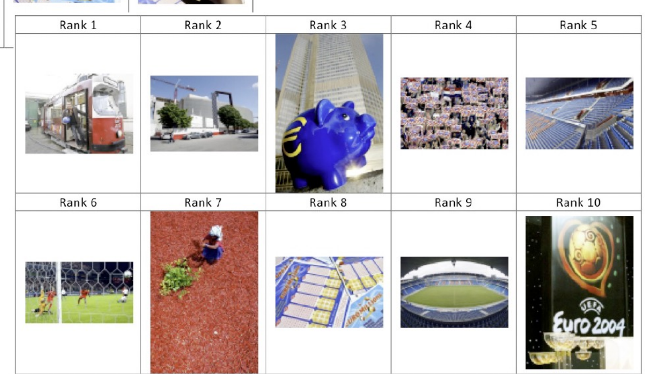
- Lots of links to Southampton University..?
  or
- Links to a range of universities

A diverse range of results has a better chance of finding what the search was actually looking for.



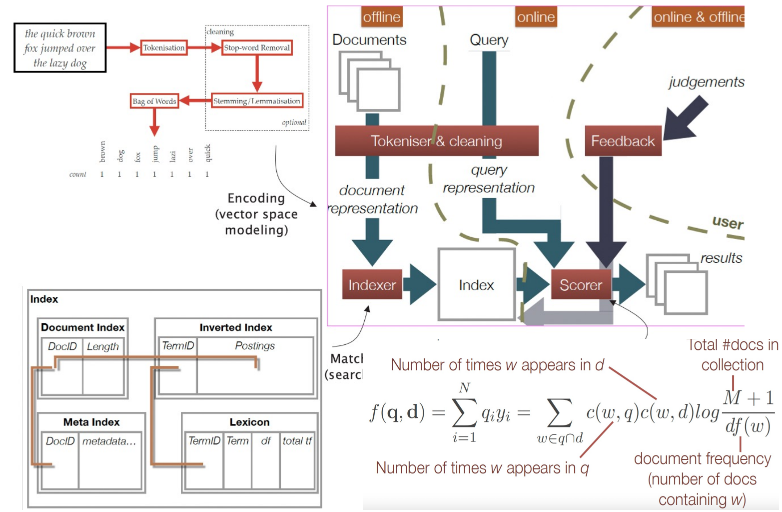
Original Ranking

Credit: Jon Hare



Diversified Ranking

Search and Rank – Summary



Search engines are a key tool in Data Mining

Important Points:

- Feature extraction
- Scalable and efficient indexing and search

Information Retrieval Process:

- Encode Documents
  - stemming, lemmatization, stop word removal
  - Make feature vector (e.g. Bag of words, TF-IDF..)
- Make index (inverted index aids search)
- Encode query
- Search index using Scoring function

Search and Rank – **Learning Outcomes**

University of Southampton

- **LO1**: Demonstrate an understanding of the fundamental concepts and approaches for search and ranking, such as: (exam)
  - ❖ Understanding the basic pipeline of searching and ranking
  - ❖ Encoding document/query using the vector space modeling methods
  - ❖ Mastering the indexing and matching methods for search
- **LO2**: Implement the learned algorithms for searching and ranking (coursework)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf*
- *ChatGPT or other AI-based techs*

Search and Rank – **Textbook**

Chapter 3

Finding Similar Items

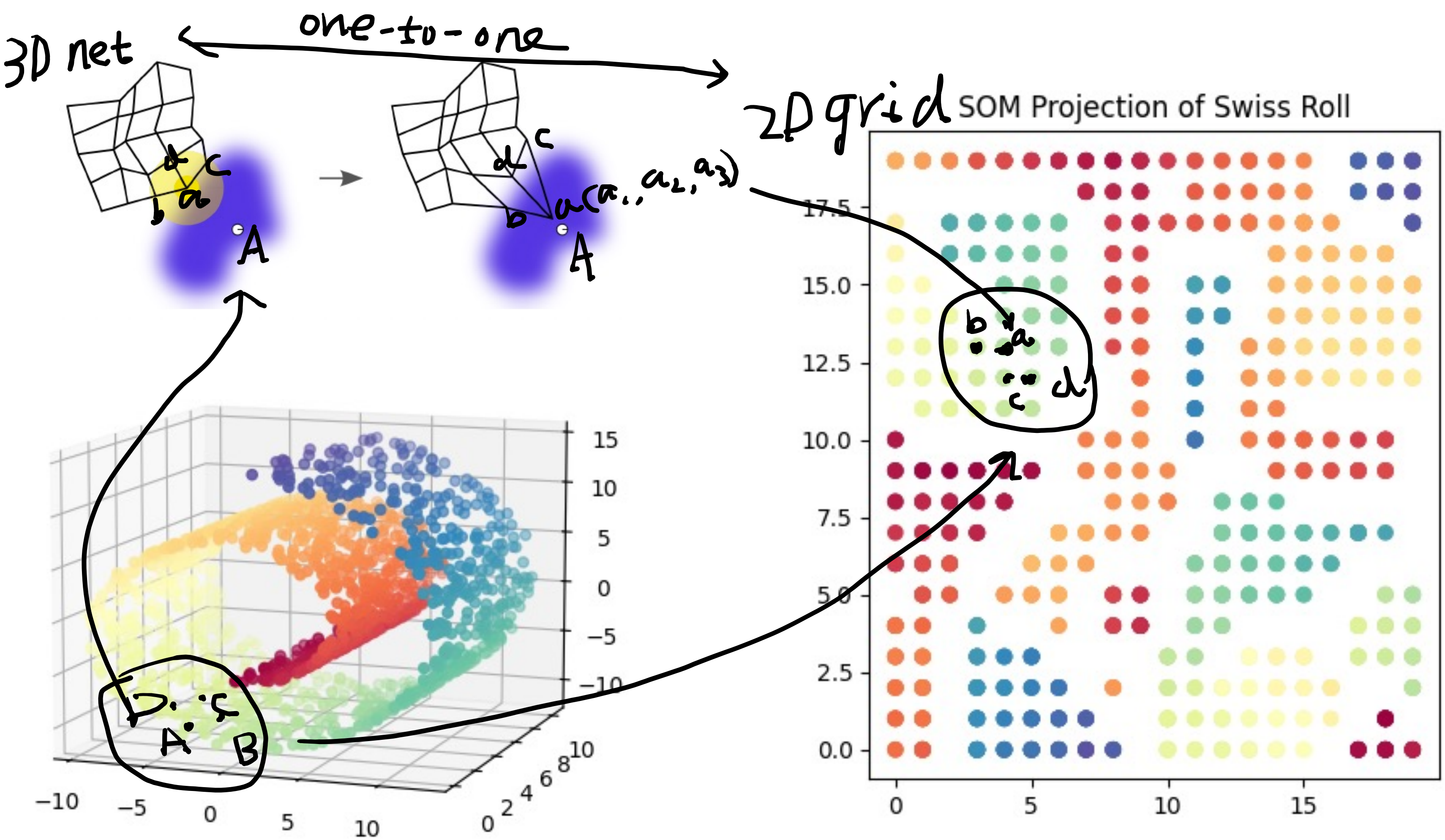
A fundamental data-mining problem is to examine data for "similar" items. We shall take up applications in Section 3.1, but an example would be looking at a collection of Web pages and finding near-duplicate pages. These pages could be plagiarisms, for example, or they could be mirrors that have almost the same content but differ in information about the host and about other mirrors.

- Mining of Massive Datasets J. Leskovec *et al*

  https://www.cambridge.org/core/books/mining-of-massive-datasets/C1B37BA2CBB8361B94FDD1C6F4E47922

Dimensionality Reduction II (recap)

SOM idea: learn local structure in high-dim space (e.g, 3D) using weight vectors, then map it onto a fixed low-dim (e.g., 2D) grid



High-dim input

Low-dim output

Dimensionality Reduction II (recap)

t-SNE idea: minimize the KL divergence between neighbor probabilities in high-dim and low-dim space, forcing local structure to be preserved.

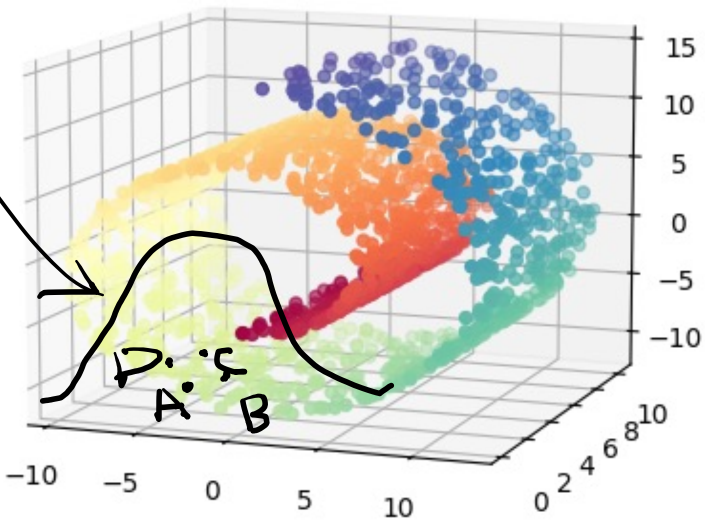
G-distribution

$$p_{j|i} = \frac{exp^{-||x_i - x_j||^2/2\sigma_i^2}}{\sum_{k \neq i} exp^{-||x_i - x_k||^2/2\sigma_i^2}}$$

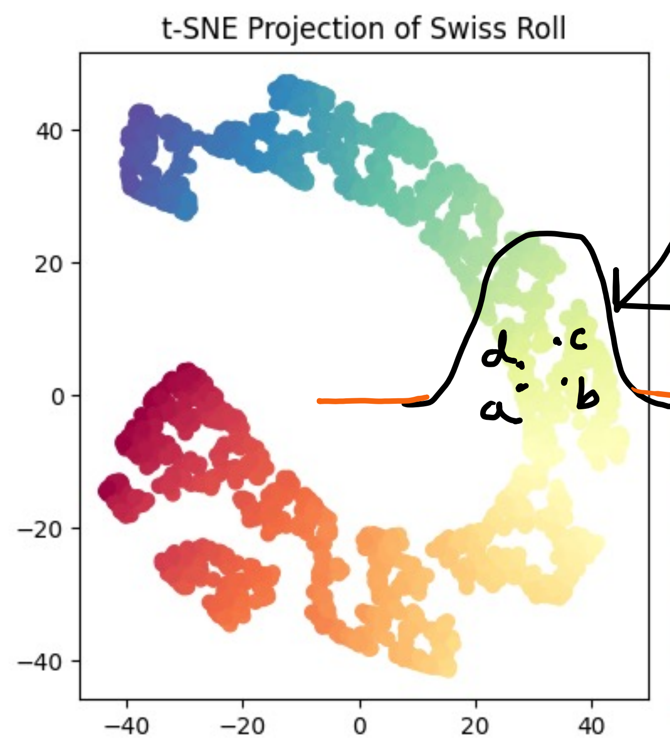
KL.

t-distribution

$$q_{ij} = \frac{(1 + ||x_i - x_j||)^{-1}}{\sum_{k \neq i}(1 + ||x_i - x_k||)^{-1}}$$



High-dim input



Low-dim output